AWS Key Management Service (KMS) & AWS Secrets Manager

What is AWS KMS?

AWS Key Management Service (KMS) is a **managed service** used to **create and manage encryption keys**.
It helps protect data by **encrypting and decrypting** it securely.

Use Cases of AWS KMS

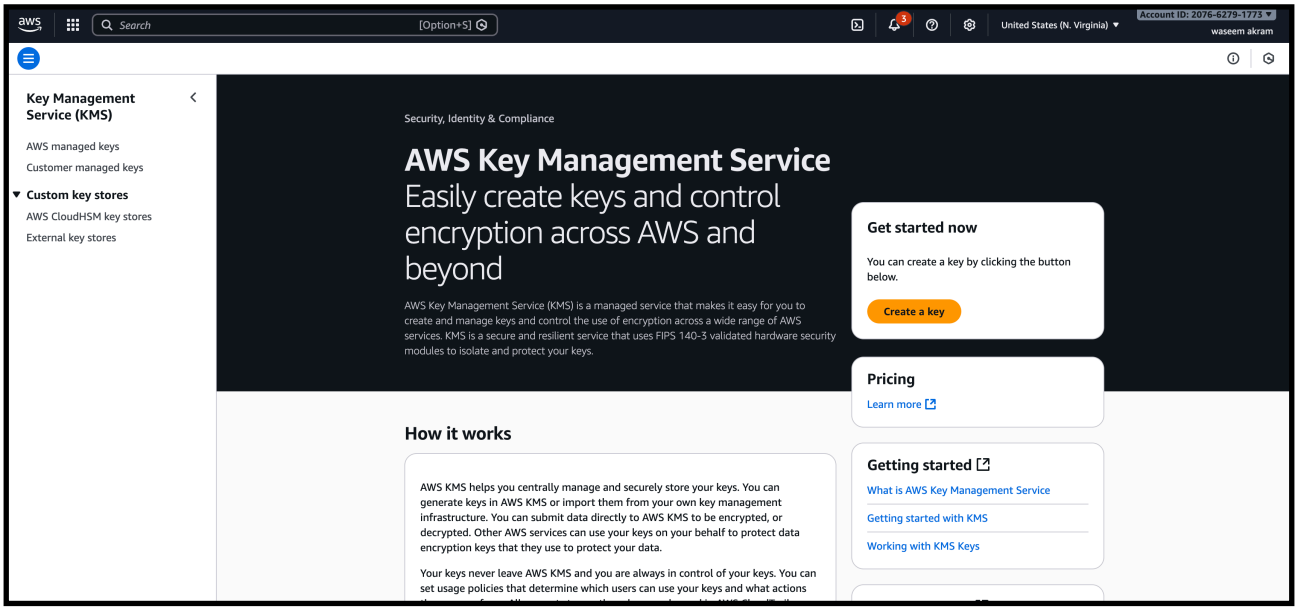
- Encrypt data stored in **S3, EBS, RDS**
- Encrypt application data
- Secure sensitive information
- Meet security and compliance requirements
- Centralized key management

Task 1: encrypt a s3 bucket using the kms key

Execution Steps – Create a KMS Key

Step 1: Open AWS KMS

- Go to **AWS Console**
- Search for **Key Management Service (KMS)**
- Click **Customer managed keys**
- Click **Create key**



Step 2: Configure Key

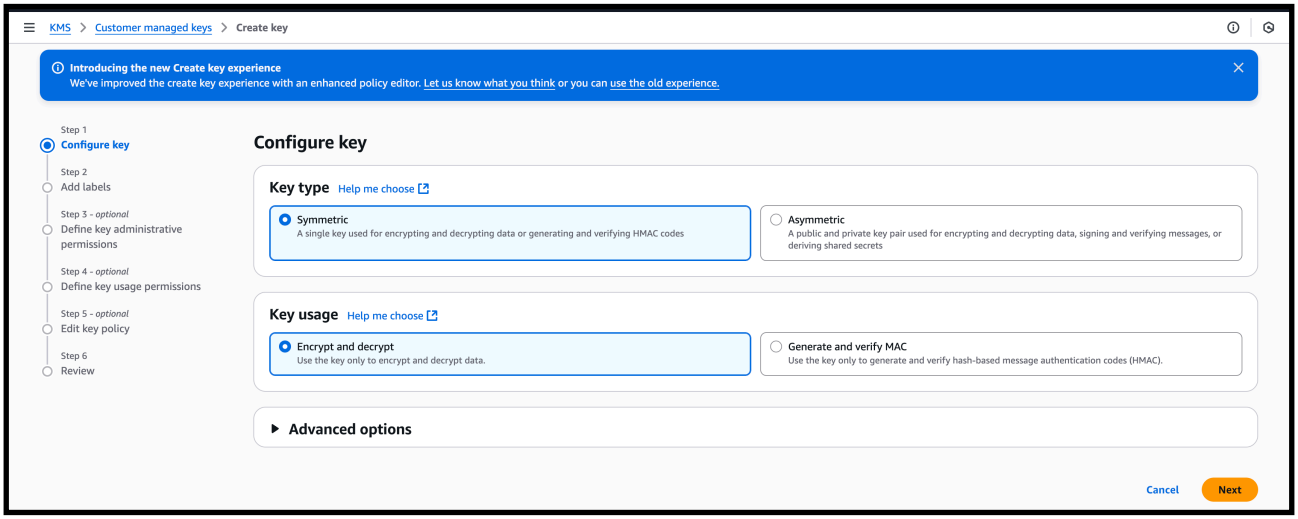
Key Type

- Select: **Symmetric**

Key Usage

- Select: **Encrypt and decrypt**

➡ Click **Next**

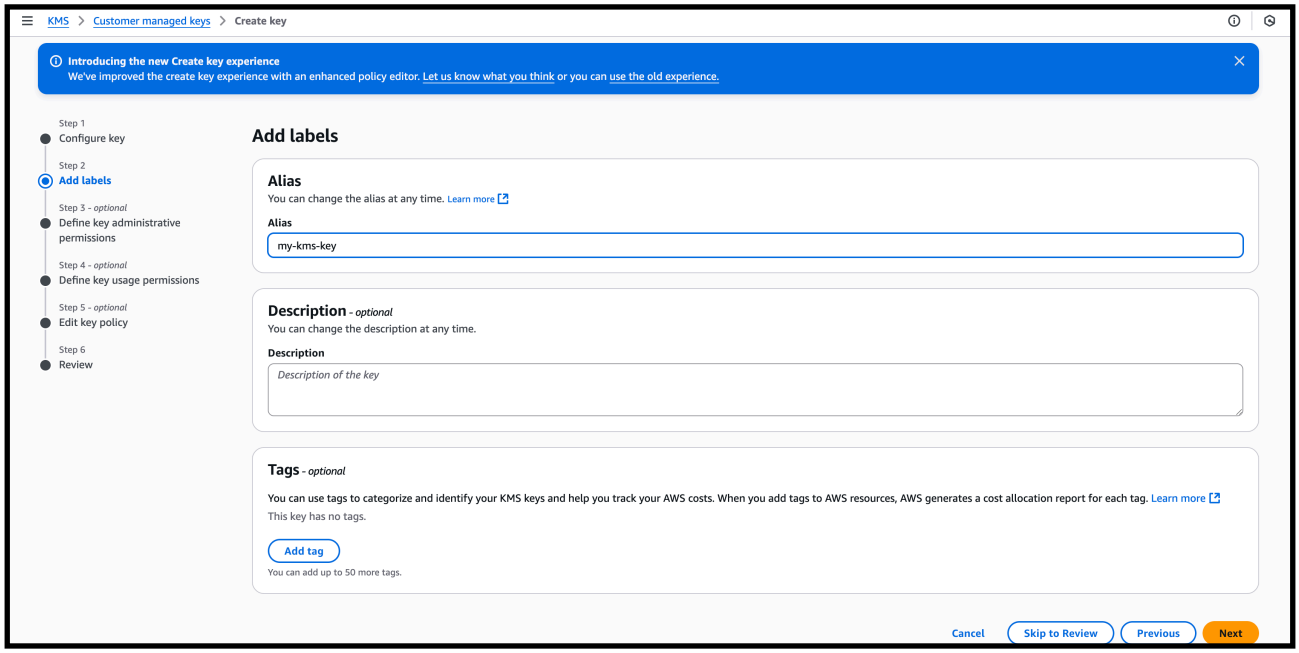


Step 3: Add Labels

Alias

- Enter alias:

```
my-kms-key
```



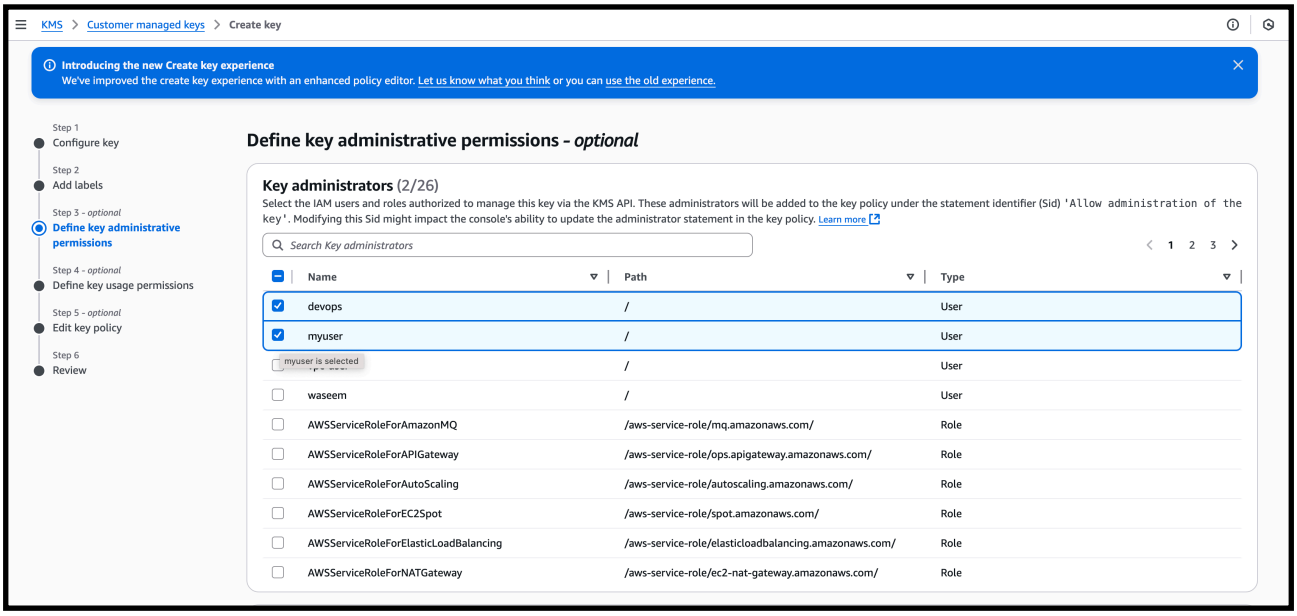
Step 4: Define Key Administrative Permissions

Select users who can **manage the key** (enable/disable, rotate, delete).

Selected:

- `devops`
- `myuser`

➡ Click **Next**

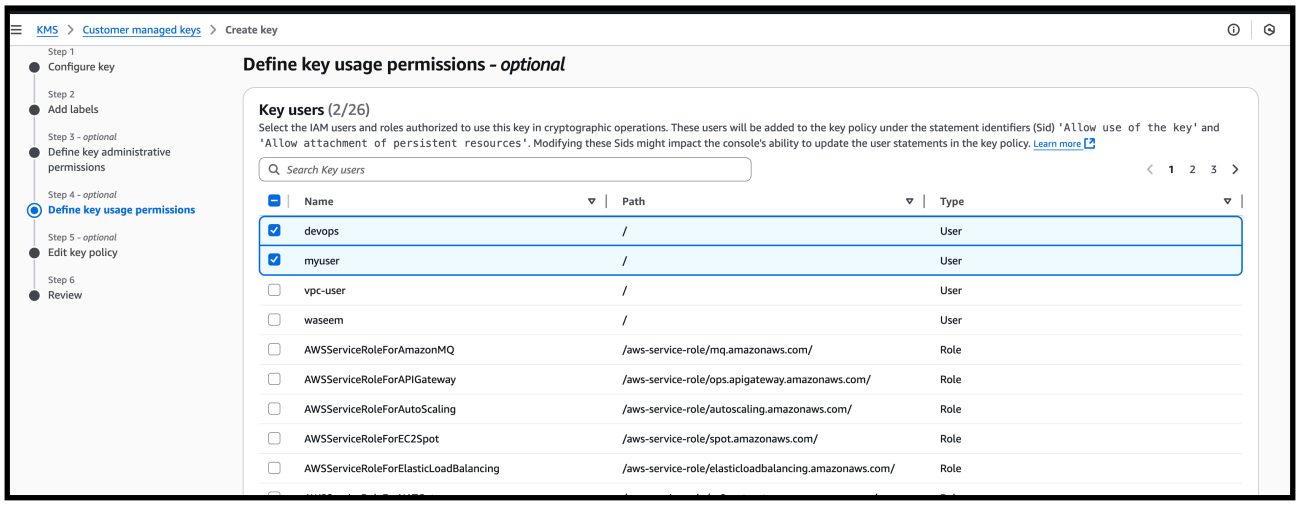


Step 5: Define Key Usage Permissions

Select users who can **use the key** for encryption and decryption.

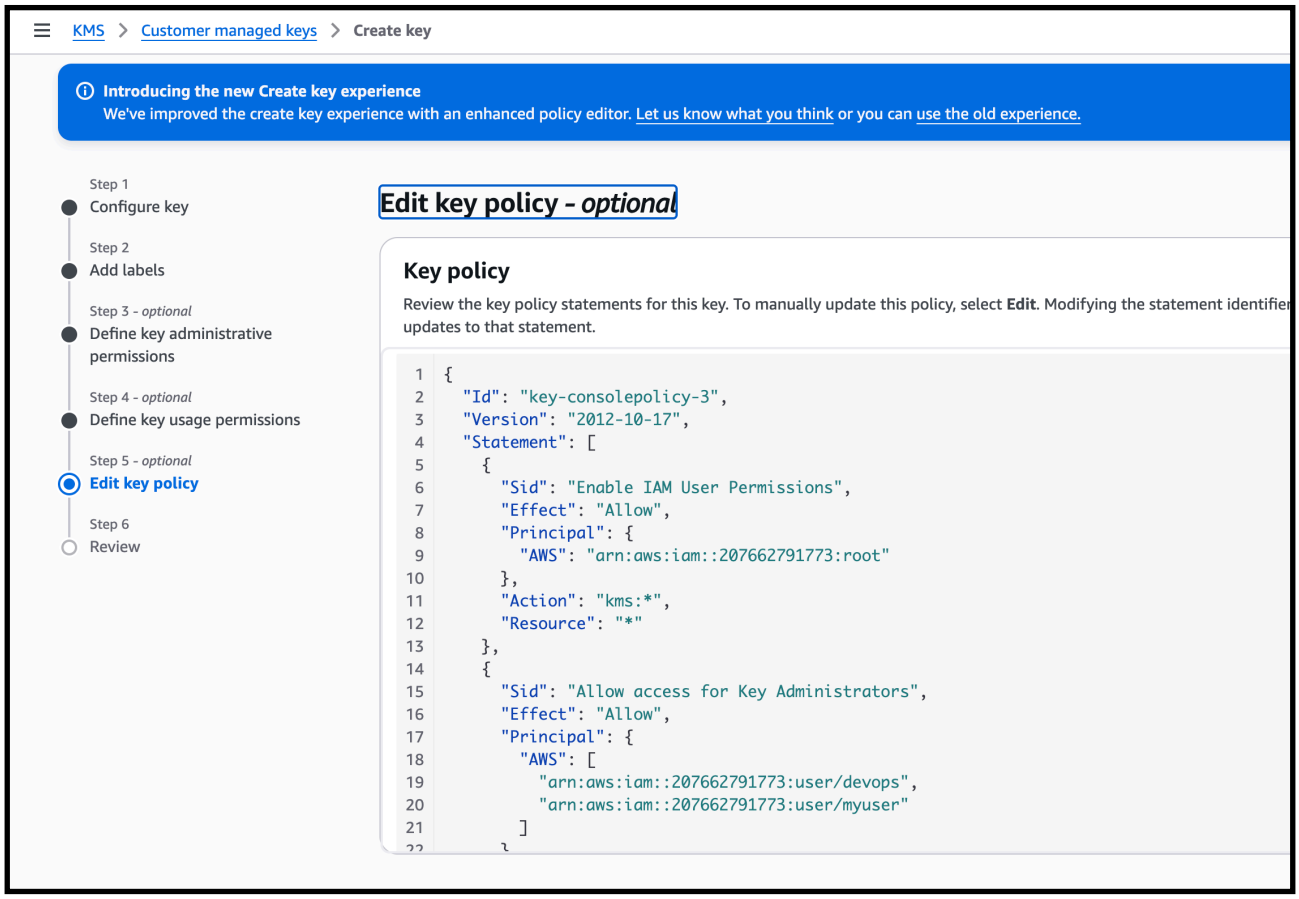
Selected:

- `devops`
- `myuser`



➡ Click **Next**

—-> keep the policy as it is



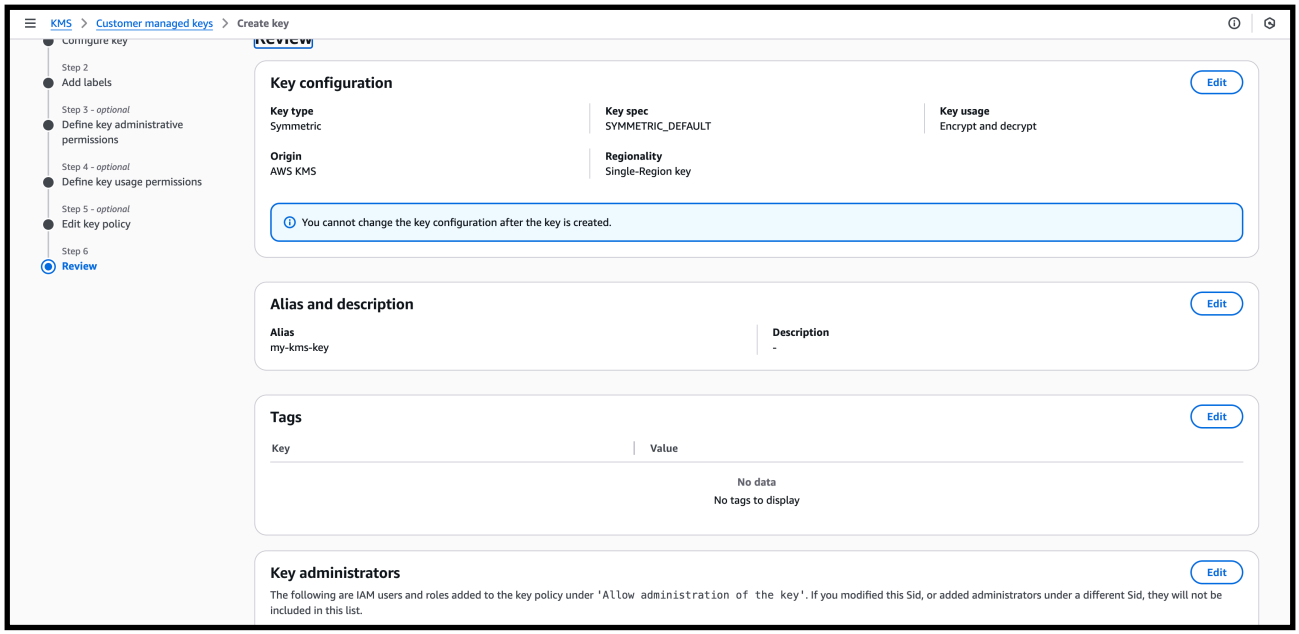
—> click next

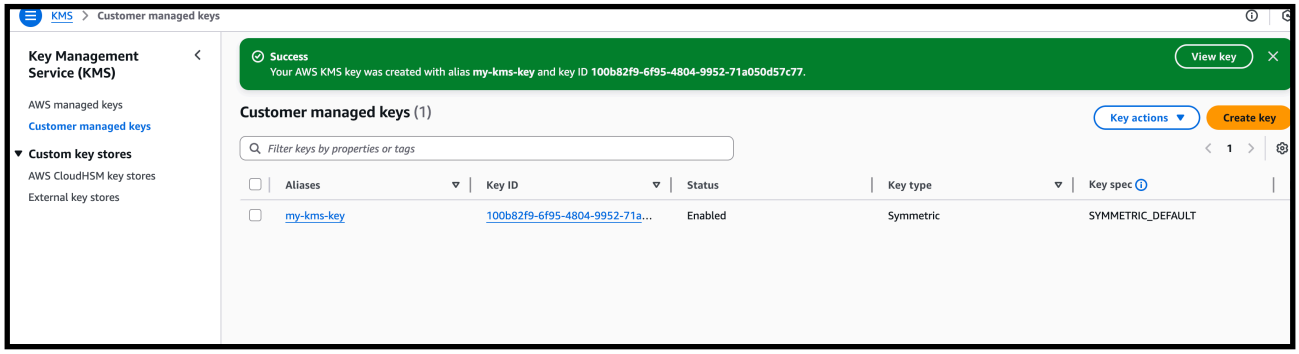
Step 7: Review and Create

Verify:

- Key type: **Symmetric**
- Key usage: **Encrypt and decrypt**
- Alias: **my-kms-key**
- Administrators: `devops`, `myuser`
- Users: `devops`, `myuser`

➡ Click **Create key**



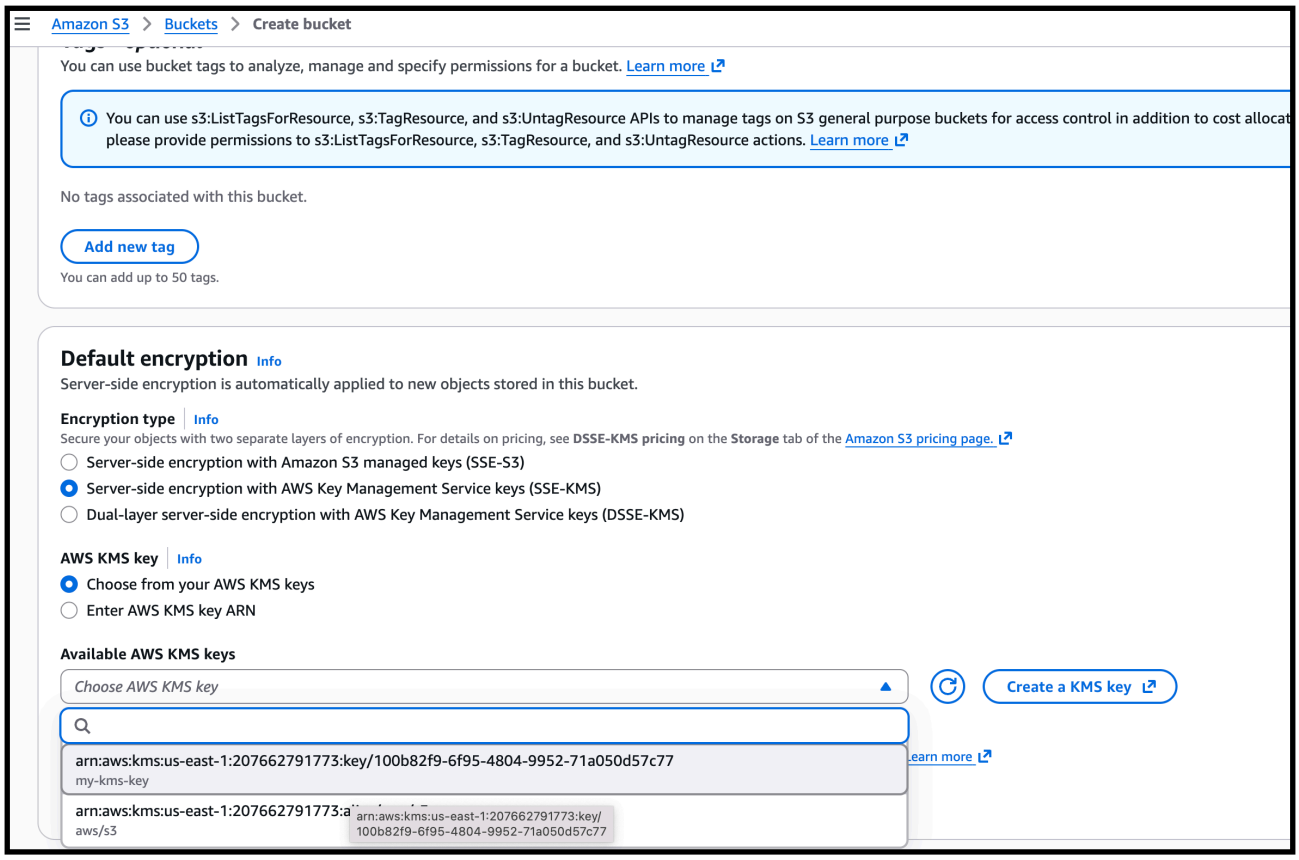


- KMS key created successfully
- Alias: **my-kms-key**
- Status: **Enabled**
- Key type: **Symmetric**
- Key spec: **SYMMETRIC_DEFAULT**

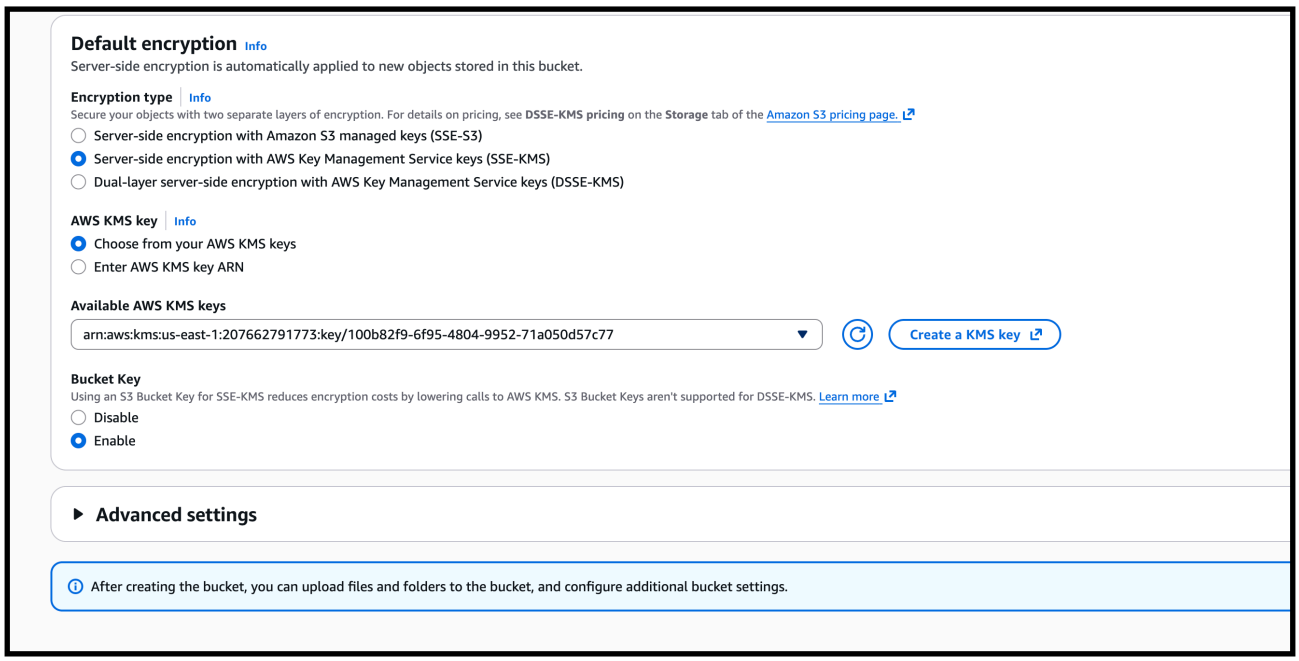
This key is now available for:

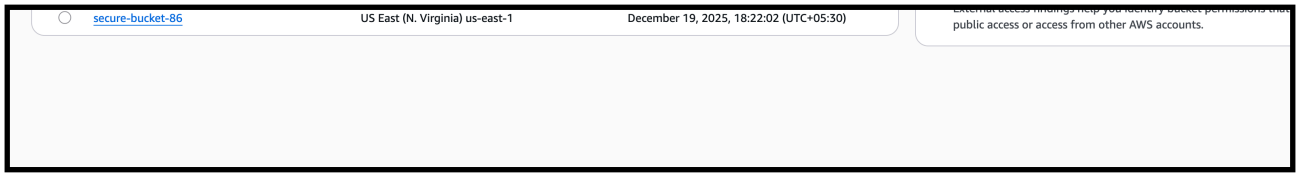
- S3 encryption
- EBS encryption
- Secrets Manager
- Other AWS services

—-> now create a s3 bucket

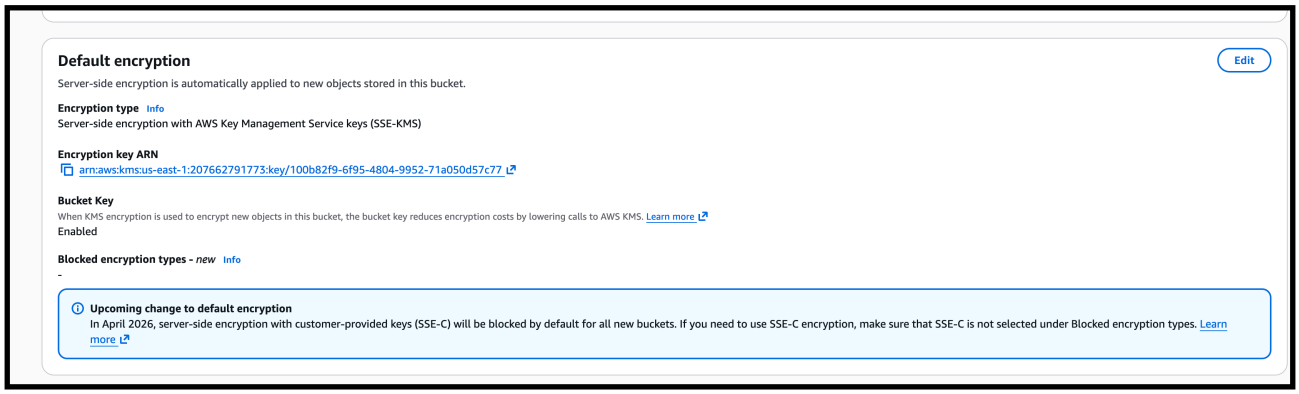


—> give the aws kms keys which we created : my-kms-key is the name of key which we created





—> s3 bucket is successfully with awk encryption role attached



Secrets manager

What is AWS Secrets Manager?

AWS Secrets Manager is a service used to **store sensitive information securely**.

Sensitive information means:

- Database username & password
- API keys
- Tokens
- Application secrets

Why we need it?
->Secrets Manager stores them **encrypted**
->Only allowed users/services can read them
->Uses **AWS KMS** for encryption

Secrets Manager = Safe locker for passwords in AWS

Use Case:

An application needs a database username and password.
Instead of storing it in code, we store it securely in **Secrets Manager**.

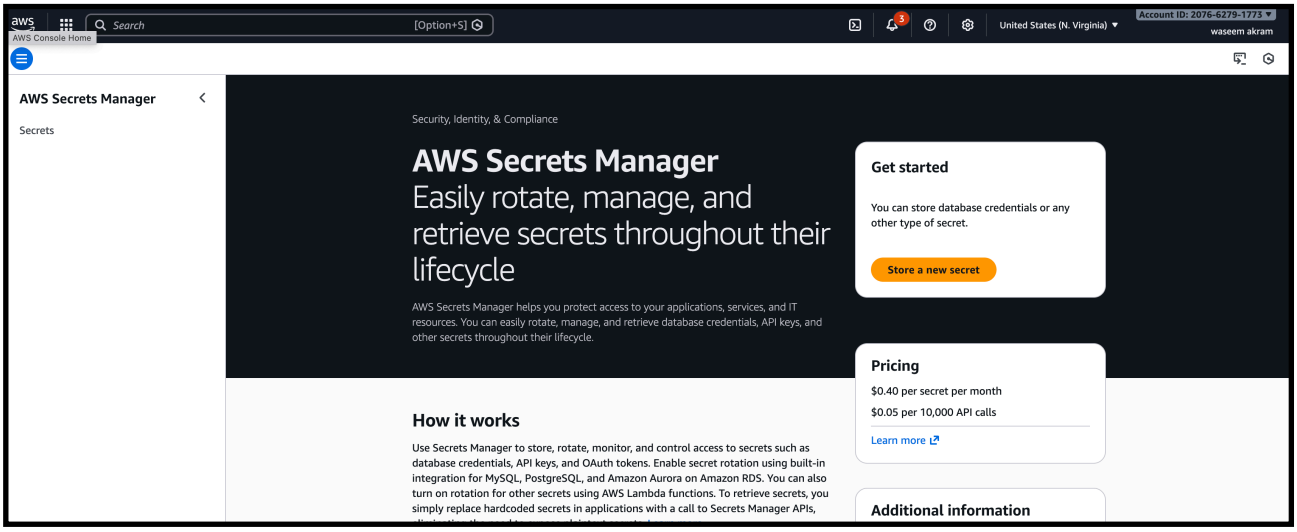
Architecture Flow (Simple)

```
User / Application
        ↓
   IAM Permission
        ↓
 AWS Secrets Manager
        ↓
      AWS KMS
```

Task: Store and Retrieve Database Credentials using AWS Secrets Manager

Step 1: Open Secrets Manager

1. Login to **AWS Management Console**
2. Search for **Secrets Manager**
3. Click **Store a new secret**



Step 2: Choose Secret Type

(As shown in your screenshot)

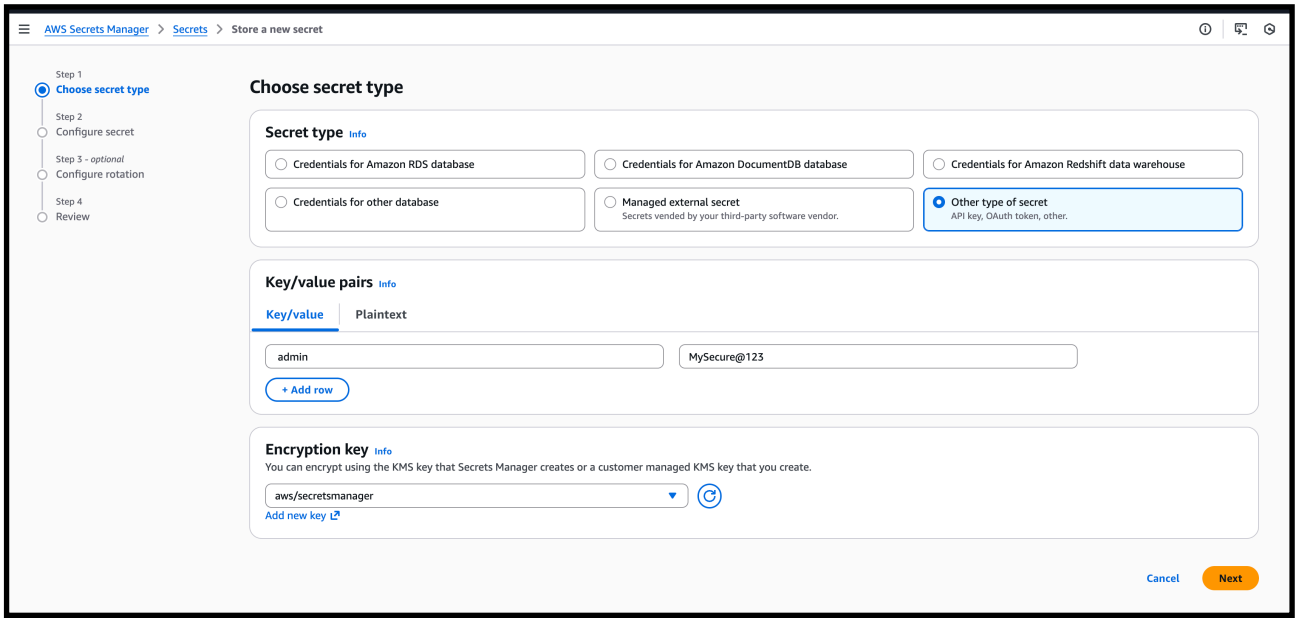
- Select **Other type of secret**
  (Used for API keys, usernames, passwords, tokens, etc.)

Step 3: Add Key / Value Pairs

Under **Key/value pairs**:

Key	Value
admin	MySecure@123

- Click **Add row** if you want to add more secrets
- Data can also be entered in **Plaintext** format (optional)



These values will be stored securely and encrypted.

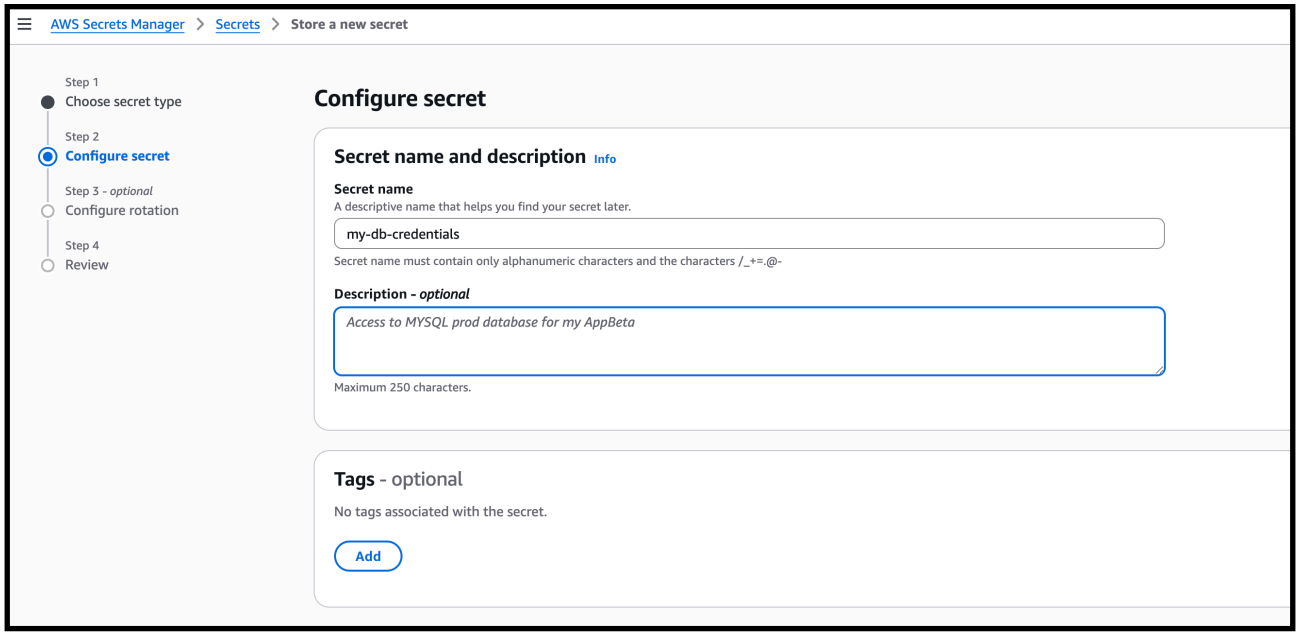
Select Encryption Key

Under **Encryption key**:

- Choose **aws/secretsmanager**
  (Default AWS managed KMS key)

This ensures the secret is **encrypted using AWS KMS**.

—> Proceed to Next Step

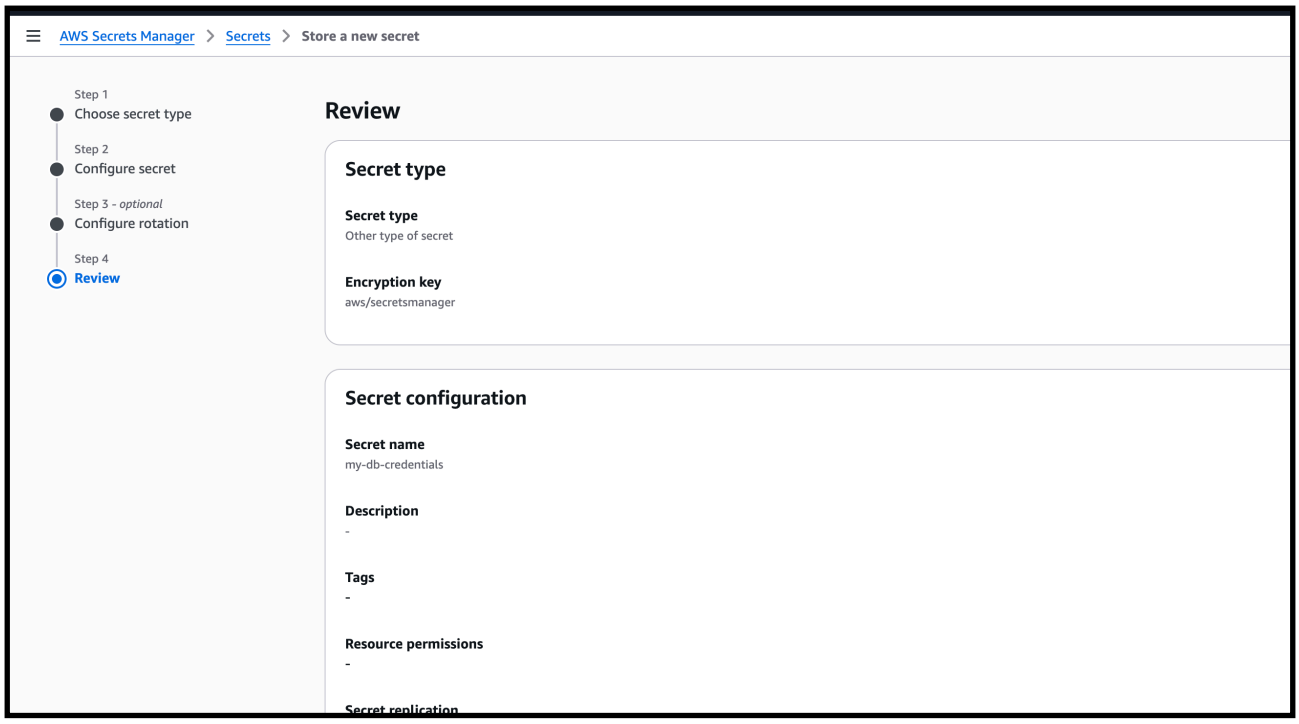


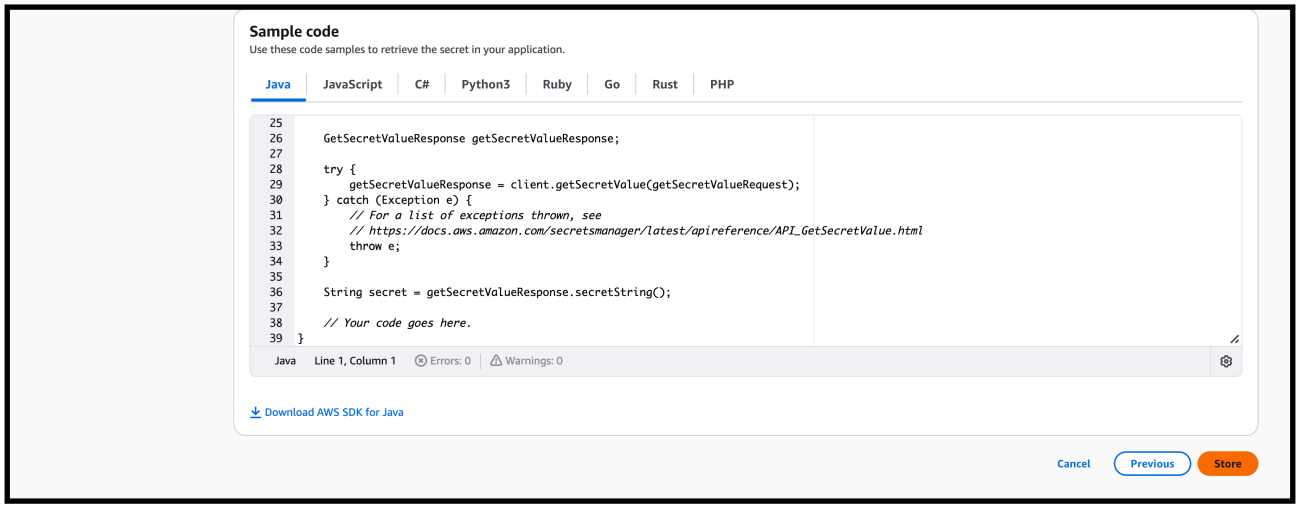
Step 4: Name the Secret

- **Secret name:** `my-db-credentials`
- **Description:** Database credentials for application

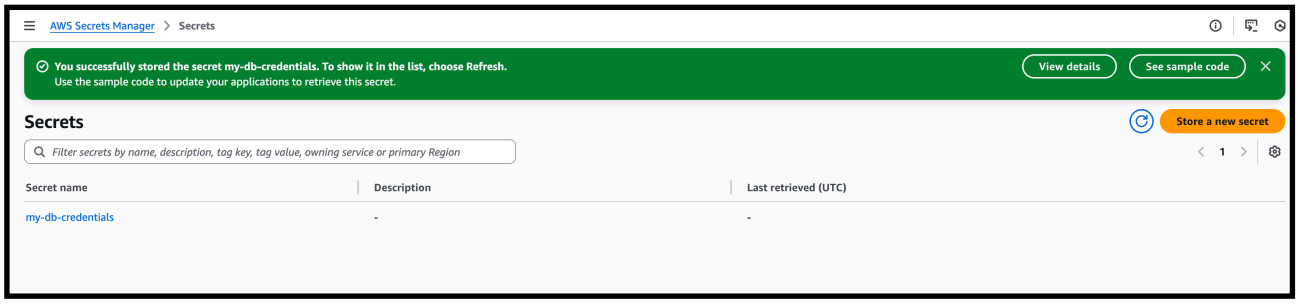
Step 5: Review and Create

- Check all details
- Click **Store**





—> successfully created a secret for aws secret manager



Step 6: Retrieve Secret (Verification)

1. Open secret **my-db-credentials**
2. Click **Retrieve secret value**

You will see:

```
{
  "username": "admin",
  "password": "MySecure@123"
}
```